https://github.com/AleixMolins3/lab1_impedance_mismatch.git

Laboratorio 1: Evaluación del Impedance Mismatch en Datos Vectoriales

1.- Introducción y Contexto del Experimento

Este documento presenta los resultados y las conclusiones obtenidas durante el laboratorio de evaluación de rendimiento de diferentes modelos de bases de datos frente al almacenamiento y la consulta de datos multidimensionales (*embeddings*).

El objetivo principal de este laboratorio es comprender y experimentar el concepto de *impedance mismatch* resultante de la ineficiencia del uso de las clásicas bases relacionales para gestionar información cuya naturaleza estructural no se ajusta al modelo, en concreto estudiando el almacenamiento y procesado de *embeddings* comparando su rendimiento con su implementación en una base de datos vectorial nativa, que se asemeja mejor a la naturaleza de estos datos debido a su arquitectura interna.

Toda la experimentación se ha llevado a cabo en un entorno Linux (WSL), utilizando Python y los motores de base de datos PostgreSQL (para el enfoque relacional e híbrido con la extensión pgvector) y Chroma DB (como representante nativo de las bases de datos vectoriales).

2.- Preparación del Entorno y del Corpus

Durante la fase inicial de obtención de datos, detectamos que el conjunto bookCorpus especificado en el enunciado ha sido retirado de los repositorios públicos de Hugging Face por cuestiones de licencia, lo que provocaba errores de URIs inválidas al intentar automatizar su descarga.

Decisión de diseño: Para respetar el objetivo pedagógico de la práctica y garantizar la calidad del proceso, optamos por extraer un corpus de texto libre alternativo (el libro Moby Dick vía Project Gutenberg).

El texto original se limpió y fraccionó sistemáticamente en frases, extrayendo una muestra final de 6.913 registros. Esta cantidad se ajusta perfectamente a la recomendación metodológica de mantener un volumen de datos manejable (alrededor de las 10.000 filas)

https://github.com/AleixMolins3/lab1_impedance_mismatch.git

para evitar latencias excesivas y un agotamiento innecesario de la memoria RAM durante la experimentación.

A partir de este punto, todas las métricas obtenidas representan el coste transaccional exclusivo de las bases de datos, ya que la inferencia del modelo de Machine Learning para generar las matrices se aisló de la medida temporal en todos nuestros scripts.

https://github.com/AleixMolins3/lab1_impedance_mismatch.git

3.- Fase P: PostgreSQL (Enfoque Relacional)

En esta fase forzamos al motor relacional a almacenar los embeddings como listas primitivas (tipo REAL[]) y a realizar consultas utilizando funciones matemáticas delegadas al backend de Python. Este proceso se realiza mediante un bucle de INSERTs a la base de datos, lo que penaliza la velocidad global. Los tiempos registrados (en segundos) son los siguientes:

	T. Min (ms)	T. Max (ms)	T. Media (ms)	Desviación Est. (ms)
P0 - Texto	1.535	9.248	1.951	0.261
P1 - Embeddings	3.000	39.266	4.216	0.839
P2 - Euclidiana	501.750	1713.573	1414.638	349.005
P2 - Cosinus	1233.636	1894.868	1672.179	229.704

Análisis de los Resultados e "Impedance Mismatch" [PQ1]:

Estabilidad de las consultas y diferencias entre métricas: Los tiempos registrados presentan una gran estabilidad (desviaciones estándar inferiores a 0.01s). Observamos que la distancia Coseno (0.123s de media) es significativamente más lenta de calcular que la Euclidiana (0.089s). Esta variación se debe exclusivamente a la complejidad del algoritmo delegado al procesador externo, que además se empeora debido a la inoperabilidad de los vectores guardados como REAL[], que nos obliga a extraerlos completamente a memoria RAM antes de procesarlos; la métrica coseno exige calcular el producto escalar y normalizar vectores de 384 dimensiones calculando raíces cuadradas sucesivas, en contraposición a las sumas de diferencias simples de la distancia euclidiana.

El desajuste de impedancia ("Impedance Mismatch"): El verdadero cuello de botella arquitectónico no es la diferencia entre métricas, sino dónde se calculan estas. PostgreSQL puro no entiende que un array de REAL[] es una coordenada espacial y no dispone de operadores geométricos nativos. Consecuentemente, para evaluar la vecindad de una sola frase (fase P2), nos vimos obligados a extraer la totalidad de la base de datos a la memoria RAM (Python) para resolver la operación secuencialmente con Scipy.

Colapso de la indexación: Esta extracción masiva (SELECT absoluto) destruye completamente la utilidad de una base de datos. Los mecanismos tradicionales de búsqueda rápida como los B-Trees pierden su sentido y el rendimiento decae dramáticamente porque se debe realizar un escaneo lineal (O(N)) para cada nueva consulta. Además, forzar matrices en el sistema incrementa casi un 71% el tiempo de inserción base.

https://github.com/AleixMolins3/lab1_impedance_mismatch.git

Decisiones de diseño: Para poder hacer uso de la librería time y extraer métricas estadísticas precisas (mínimo, máximo y desviación estándar) para cada documento, diseñamos los scripts ejecutando una sentencia INSERT acompañada de un commit() dentro de un bucle secuencial. Esta decisión arquitectónica se traduce en 6.913 llamadas individuales a la base de datos. Aunque a nivel de código supone una estructura sencilla, el alto número de llamadas penaliza drásticamente el rendimiento global debido a la latencia transaccional de red repetida en cada iteración (patrón clásico OLTP), una penalización que asumimos conscientemente para garantizar la precisión de la recogida de datos a nivel de fila.

Alternativas de inserción e indexación en PostgreSQL puro [PQ1]: Evaluando si existen métodos o estructuras nativas para mejorar el rendimiento sin utilizar pgvector, hemos llegado a las siguientes conclusiones técnicas:

Método de inserción: El rendimiento de carga podría haber mejorado exponencialmente sustituyendo los INSERT individuales por el comando nativo COPY de PostgreSQL (o la función execute_values de psycopg2), diseñada para cargas masivas (bulk load). Sin embargo, esto nos habría impedido medir la desviación estándar de cada transacción individual, tal como exigía el experimento.

Estructuras de datos e indexación: Aunque PostgreSQL dispone de índices avanzados como GIN o GiST (óptimos para texto completo o geometrías 2D/3D), ninguno de estos índices tradicionales soporta operaciones algebraicas de similitud sobre listas de alta dimensionalidad (REAL[]). Adicionalmente, extensiones clásicas como cube están limitadas por hardware a 100 dimensiones, siendo inviables para nuestros vectores de 384 dimensiones. Consecuentemente, sin pgvector, ninguna técnica de indexación puede evitar el escaneo secuencial completo, confirmando que el desajuste de impedancia geométrico es insalvable a nivel estructural puro.

https://github.com/AleixMolins3/lab1_impedance_mismatch.git

4.- Fase C: Chroma (Enfoque Vectorial Nativo)

En la segunda fase utilizamos Chroma DB, una herramienta diseñada desde cero para almacenar, indexar y consultar conjuntos de datos vectoriales densos, mitigando el desajuste de la naturaleza de los conjuntos de datos vectoriales y su procesamiento. Los resultados de rendimiento obtenidos son los siguientes:

	T. Min (ms)	T. Max (ms)	T. Media (ms)	Desviación Est. (ms)
C0 - Texto	208.127	1502.113	485.665	293.142
C1 - Embeddings	99.835	461.062	182.734	91.010
C2 - Euclidiana	2.746	5.171	3.387	0.720
C2 - Cosinus	2.769	19.050	4.937	4.991

Análisis de los Resultados vectoriales [CQ1]:

Evaluación separada de datos y texto: Uno de los retos de esta fase fue responder si era posible medir la inserción solo del texto cuando trabajamos con Chroma, ya que el motor autovectoriza los documentos por defecto. Concluimos que sí es posible si se engaña a la arquitectura: en la fase C0, inyectamos *arrays* rellenados con ceros (dummy vectors) manualmente. Así evitamos la ejecución del modelo interno, demostrando que el coste de almacenar el documento por sí solo cae hasta los 0.002s.

El Impedance Mismatch inverso y la reducción de llamadas: A pesar de ser rapidísima con los vectores, sufrimos el efecto contrario al de PostgreSQL. Al intentar copiar el patrón relacional de INSERTs, el sistema colapsa debido a la sobrecarga de la generación del grafo interno, debido a las 6.913 llamadas e inserciones individuales (fila a fila). Esta situación nos forzó a tomar una decisión de diseño crítica: reestructuramos el código añadiendo lógica de partición para enviar los datos en bloques (*batches*) de 100 registros. Este incremento en la complejidad y líneas de código permitió reducir drásticamente el número de llamadas a la API de 6.913 a solo 70 peticiones, siendo la única estrategia viable para estabilizar el motor vectorial y hacer que el rendimiento fuese sostenible.

Escalabilidad extrema en las consultas (C2): Los resultados de C2 demuestran por qué existen estas soluciones. El cálculo del Coseno pasó de tardar 0.123s en la fase P2 a solo 0.0059s en esta fase (20 veces más rápido). La ausencia de desajuste geométrico permite al motor indexar los vectores de forma nativa mediante algoritmos de búsqueda aproximada (HNSW), recuperando los vecinos similares navegando por un espacio preorganizado en lugar de escanear 6.913 registros uno a uno.

https://github.com/AleixMolins3/lab1_impedance_mismatch.git

Alternativas de inserción e indexación en Chroma [CQ1]: Respondiendo a la cuestión sobre métodos o estructuras para mejorar el rendimiento en Chroma, hemos determinado lo siguiente:

Método de inserción: Aparte de la estrategia de inserción por bloques (*batching*) que ya implementamos para evitar el colapso del sistema, optimizar y ajustar matemáticamente el tamaño de este bloque (*batch size*) permitiría maximizar la eficiencia, reduciendo aún más el *overhead* de la red y minimizando el I/O en escrituras masivas.

Técnicas de indexación: La estructura de datos central que utiliza Chroma para resolver las distancias es el algoritmo basado en grafos HNSW. Para alterar el rendimiento de estos operadores, se pueden afinar sus hiperparámetros internos en el momento de crear la colección. Por ejemplo, si se reducen los valores de M (máximo de conexiones por nodo en el grafo) y ef_construction (tamaño de la lista de candidatos dinámicos), se acelerará significativamente el tiempo de construcción del índice (reduciendo los tiempos de inserción) a costa de sacrificar ligeramente la precisión (*recall*) durante las consultas de similitud. Este compromiso (*trade-off*) estructural es clave para balancear el rendimiento y escoger entre velocidad de escritura o exactitud de búsqueda.

https://github.com/AleixMolins3/lab1_impedance_mismatch.git

5.- Fase G: Pgvector (Enfoque Híbrido)

La fase final consistió en dotar a nuestra base de datos relacional original de capacidades algebraicas puras mediante la extensión pgvector, permitiendo usar el tipo VECTOR(384) de forma nativa y los algoritmos y funciones matemáticas necesarias asociadas.

En este caso, la adopción del tipo vectorial nativo redujo el tiempo de inserción en un 30%, mientras que el poder emplear los operadores propios SQL para el cálculo de la Euclidiana y Cosinus de forma interna marca la diferencia principal, con unos resultados similares al rendimiento bruto visto en Chroma DB.

	T. Min (ms)	T. Max (ms)	T. Media (ms)	Desviación Est. (ms)
G0 - Texto	2.428	26.615	3.163	0.773
G1 - Embeddings	3.196	17.888	4.325	0.725
G2 - Euclidiana	4.310	18.409	6.107	4.266
G2 - Cosinus	4.608	6.713	5.395	0.700

Análisis de los Resultados híbridos:

La simple activación del tipo nativo en el esquema (fase G1) supuso una reducción del tiempo de inserción del 30% en comparación con los intentos primitivos con matrices (fase P1). No obstante, el cambio estructural más crítico se puso de manifiesto durante la consulta G2. Haciendo uso exclusivo de instrucciones SQL con los nuevos operadores nativos (<-> para Euclidiana y <=> para Coseno), el motor puede calcular el álgebra lineal y ordenar los resultados de forma interna.

La resolución del conflicto: Hemos podido abaratar la consulta en coseno de 0.123s a solo 0.005s, equiparando e incluso mejorando marginalmente el rendimiento puro de Chroma DB. Este enfoque híbrido elimina completamente el desajuste de impedancia y el cuello de botella de red/memoria, permitiéndonos disfrutar de la robustez ACID relacional sin sacrificar el potencial vectorial.

El impacto del escaneo secuencial nativo: Lo más destacable de esta resolución es que los tiempos de consulta geométrica se han desplomado hasta los 0.005s sin necesidad de construir un índice vectorial avanzado (como HNSW o IVFFlat) sobre la tabla. Esto demuestra que el desajuste de impedancia original no era por el escaneo secuencial en sí (ya que con pgvector seguimos haciendo un Sequential Scan de las 6.913 filas), sino por el coste de serialización, la transferencia por red hacia el backend y la ejecución matemática interpretada en Python. Llevar el operador nativo al espacio de memoria de PostgreSQL y

https://github.com/AleixMolins3/lab1_impedance_mismatch.git

ejecutarlo en C ha sido suficiente para igualar el rendimiento de una base de datos nativa vectorial para este volumen de datos.

Finalmente, los resultados demuestran que la elección de la herramienta óptima depende intrínsecamente de la arquitectura de datos del proyecto. Las bases vectoriales puristas (Chroma) son excepcionales a la hora de buscar, pero presentan latencias graves y cuellos de botella transaccionales cuando se exigen inserciones aisladas. Por su parte, el enfoque híbrido permite fusionar lo mejor de ambos mundos de forma transparente.

https://github.com/AleixMolins3/lab1_impedance_mismatch.git

6.- Discusión Global

Desde la perspectiva del desajuste de impedancia, la principal diferencia consiste en cómo el motor de la base de datos procesa la información de naturaleza multidimensional, forzando los vectores matemáticos a estructuras primitivas de la base relacional, o ajustándolas con una base de datos vectorial nativa para poder realizar las operaciones de álgebra espacial internamente.

Por lo tanto, obviando el esquema que presenta PostgreSQL con sus claras desventajas, por un lado Chroma DB nos presenta una solución que elimina el desajuste geométrico de los embeddings e integra algoritmos de búsqueda aproximada, lo que dispara la velocidad de las consultas. En cambio, tiene severos problemas de latencia cuando se exigen inserciones aisladas y un registro de datos fila a fila, lo que colapsa el motor al reescribir el grafo constantemente.

De igual manera, la implementación con pgvector y sus operadores resuelven el conflicto algebraico, eliminando la extracción de los datos a memoria de la base puramente relacional. Sin embargo, esta base de datos requiere una gestión más compleja de las extensiones en el entorno del servidor, y la creación de índices HNSW de forma manual que ya estaban presentes en Chroma DB, para un rendimiento similar si el volumen crece a los millones de registros.

En conclusión, la elección de la herramienta depende de la arquitectura de datos del proyecto, ya que las bases vectoriales son excepcionales para la búsqueda y cálculo masivo similar a este estudio, pero carecen de versatilidad transaccional; mientras que extensiones como pgvector fusionan lo mejor de ambos tipos eliminando la *impedance mismatch* sin necesidad de hacer un despliegue de bases de datos paralelas.

https://github.com/AleixMolins3/lab1_impedance_mismatch.git

7.- Uso de Inteligencia Artificial y Estrategia de Prompts

Para el desarrollo integral de este laboratorio nos hemos apoyado en el uso de modelos de lenguaje (LLM) actuando como copilotos técnicos, tanto para la implementación del código de medición en Linux/WSL como para el repaso conceptual. En lugar de hacer *prompts* genéricos buscando "la solución final", hemos estructurado una metodología en tres grandes líneas de actuación:

"Chain of Thought" Técnico (Secuenciación de tareas): Para asegurarnos de que respetamos los límites científicos de la recogida de datos, pedimos la estructuración de las tareas paso a paso (ej. "Haremos primero el script P0, quiero recoger estas métricas con la librería time. Una vez confirmados los tiempos, procederemos a P1..."). Esto nos evitó arrastrar errores de implementación globales si la fuente de datos hubiera fallado.

"Zero-Shot Debugging" con Trazabilidad de Contexto: El entorno presenta retos inmediatos: errores 404 de la API de *Hugging Face* por archivos obsoletos o la falta de permisos SUPERUSER para instalar extensiones SQL en Ubuntu. La estrategia fue realimentar el modelo transfiriendo íntegramente los bloques completos de *Traceback* del terminal sin filtros explicativos añadidos por nosotros. Esto dio a la IA la consciencia absoluta de las rutas y paquetes de Linux que teníamos instalados, permitiendo pivotar rápidamente hacia soluciones como la compilación *make* de pgvector directamente desde el GitHub oficial en lugar del repositorio *apt-get*.

Extracción de Insight Pedagógico y Fundamentación: El objetivo de la práctica no era obtener código funcional, sino evidenciar el Impedance Mismatch. Para extraer este valor, utilizamos de forma iterativa instrucciones metacognitivas (*"Explícame el porqué de cada línea y lo que acabamos de hacer, basándote en la teoría de bases de datos"*). El análisis de por qué la operación .tolist() en Python era la raíz de la penalización de memoria o la causa técnica del colapso de Chroma si no se utilizaban lotes (*batches*) proviene de este cuestionamiento activo de la arquitectura por nuestra parte hacia el modelo.